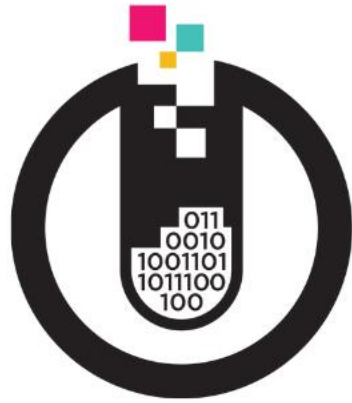


Delta Lake Optimisation

Partition & Clustering Best Practices



DATA:Scotland 2025



**THE
DATA LAB**
value from data



DATAmasterminds



**ADVANCING
ANALYTICS**



Introduction



Bas Land

- Data consultant since 2013
- I've done Power Pivot, SQL Server, Power BI, Azure SQL, and now Fabric
- Dataplatform MVP, since last week 😊
- Brazilian jiu-jitsu purple belt, weight lifting, running
- Husband and father



Connect with me

LinkedIn



YouTube



ThatFabricGuy.com



Slides and code samples

- The slides and code samples for this session will be posted to my public Github repo:
- <https://github.com/basland/presentations/>
- Also check my YouTube and blog for content like this:
 - [YouTube.com/@ThatFabricGuy](https://www.youtube.com/@ThatFabricGuy)
 - <https://thatfabricguy.com>



Performance teaser...

Optimisation	Single Date	Date Range
None	67.47 sec	64.63 sec
Partition by date	3.48 sec	3.94 sec
Cluster by date	7.88 sec	3.48 sec



Why this talk?

- Delta Tables power your Fabric data platform
- Unoptimised Delta = sluggish, expensive, frustrating!
- Modern data engineers must think **physical**
- Delta Lake in Fabric gives us powerful tools & complexity...



What you'll learn today

- Partitioning: get partition pruning for free
- Liquid Clustering: continuous optimisation on write
- Live Benchmarks: partitioned vs clustered vs unoptimized
- File Compaction & VACUUM: managing the small files beast
- Best Practices



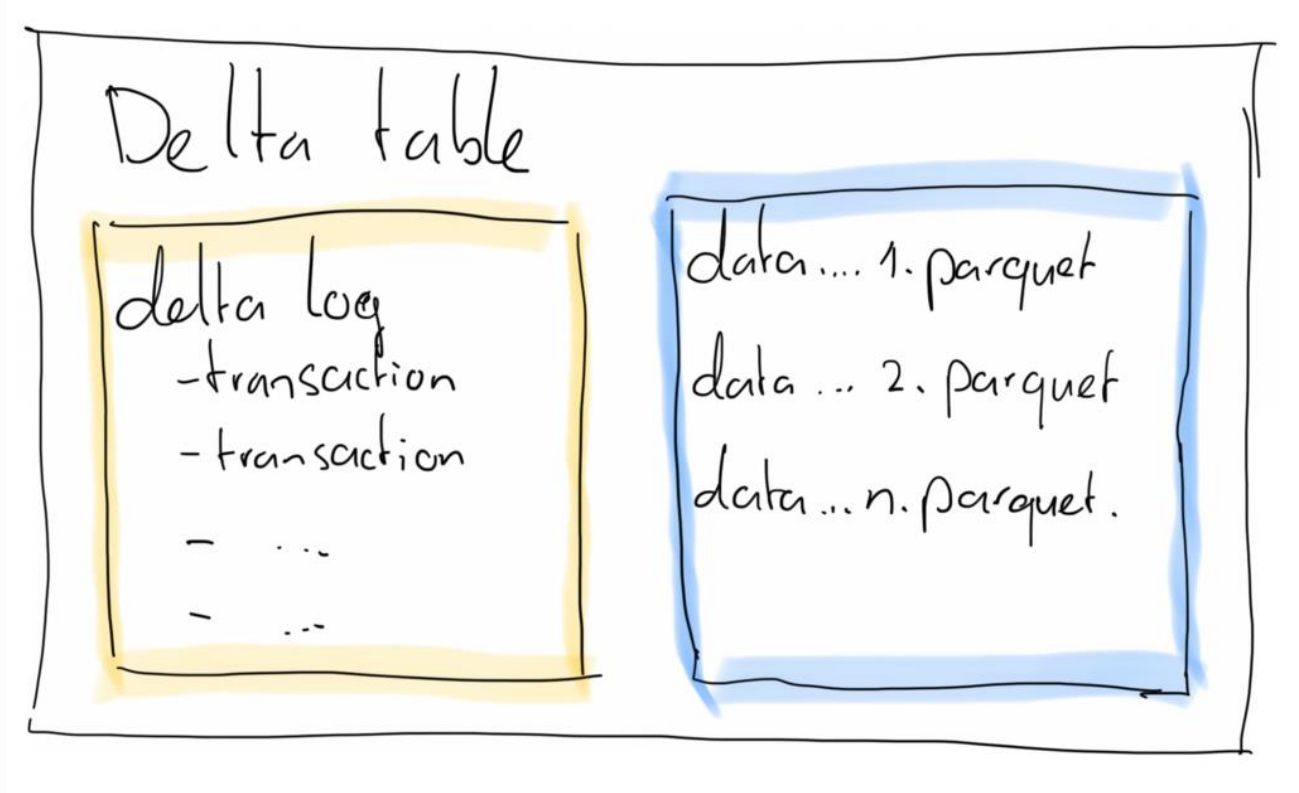
Delta Lake Anatomy

& optimisation principles



What is a Delta Table?

- Delta = Parquet + Transaction
- ≥ 1 parquet files
- + transaction log
- + ACID transactions
 - Merge, update, delete
 - (atomic, consistent, isolated, durable)



How queries work

- Delta table = files to be queried as tables
- Query execution:
 1. SQL/Spark query submitted to cluster
 2. Delta log is read for latest snapshot
 3. Min/max stats used to prune (select) files
 4. Relevant parquet files are scanned (only relevant columns)
 5. Result set is prepared and presented (to client, or to data frame, or...)



What are we optimising for?

Goal	Why it matters
Fewer files scanned	Less I/O = faster queries
Smaller file sizes	Faster read (better caching)
Better file layout	Enables skipping on multiple columns
Lower shuffle volume	Efficient joins and aggregations

Delta table

delta log

- transaction

- transaction

- ...

- ...

data ... 1. parquet

data ... 2. parquet

data ... n. parquet.



Fabric-specific enhancements

- Fabric gives us some freebies!
- **V-Order**: VertiPaq-friendly (for DAX) parquet layout
- **DirectLake**: Power BI queries delta tables without import (layout matters even more with **many many many** read queries)
- V-Order is enabled by default.
Layout optimisation is still **your job!**



Optimisation toolbox

- Four tools for faster Delta Tables
 1. Partitioning
 2. Liquid Clustering
 3. Compacting files (OPTIMIZE)
 4. Removing obsolete files (VACUUM)

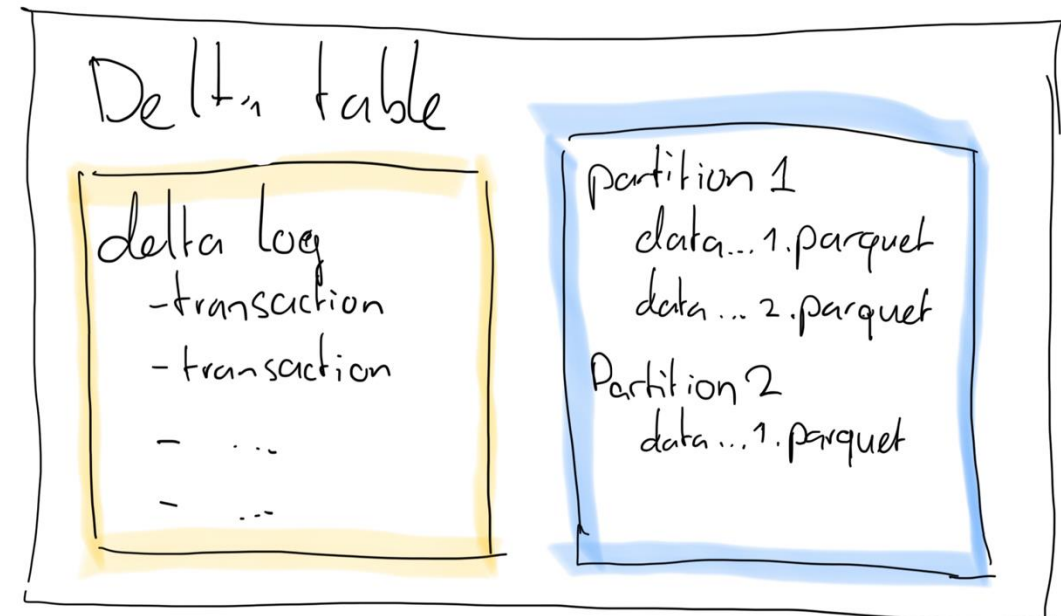


All about Partitioning



Partitioning – classic optimisation

- Physically split data in folders (force multiple parquet files)
- Enables **partition pruning**: only read relevant files
- Perfect for **very large tables**
- Partition on **columns used in filters**



Pitfalls of over-partitioning

- Too many partitions = too many small files
- Queries without partition in filter fetch **ALL files**
- Unnecessary for small and medium tables



When to use partitioning



Very large tables

Partitioning is a great optimisation strategy for very large tables, think 100 million rows and up



Predictable filter patterns

Partitioning speeds up read queries that utilise partition columns in their filters



Introducing Liquid Clustering



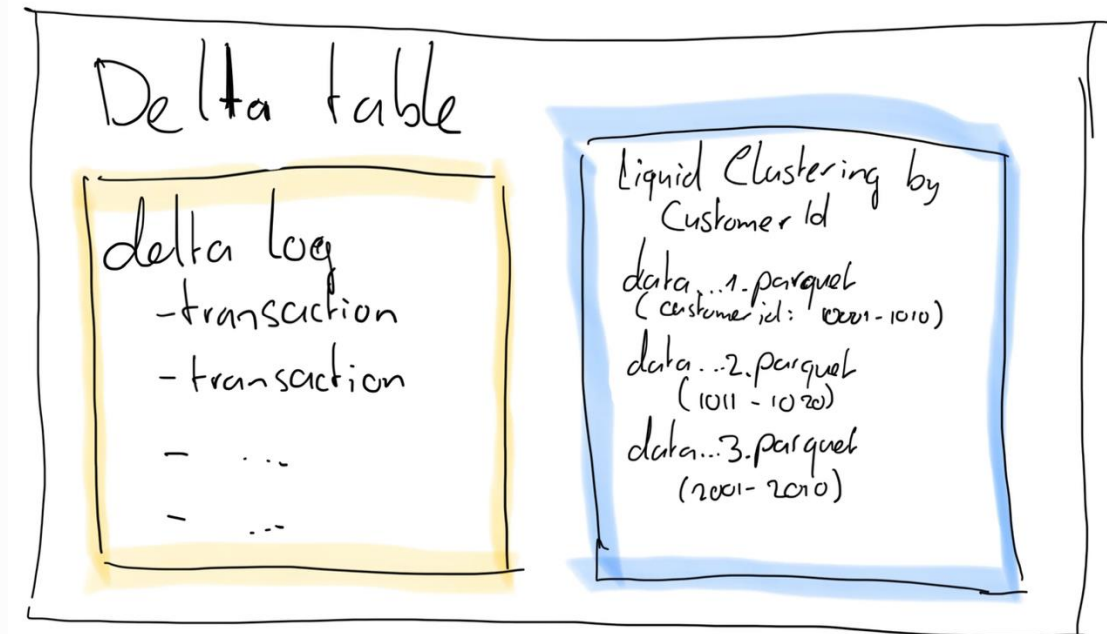
The limits of partitioning

- Works only for fixed set of columns
- Bad in high-cardinality columns
- Queries without partition filter still scan all files
- Requires fixed schema, difficult to maintain and change later



What is Liquid Clustering?

- Organises data by columns without physical file splits
- Applied incrementally during write, no maintenance job needed
- Rows with similar values end up in the same files
- Helps data skipping on read for filtered queries



Clustering vs V-Order

Feature	V-Ordering	Liquid Clustering
How to apply?	Batch (run OPTIMIZE command)	Incremental on write
Compute cost	High (sort + write)	Low to moderate
Maintenance implication	Yes, periodically (e.g. weekly)	No, clustering is automatic
Flexibility	High, but manual	High, automatic



When to use Liquid Clustering

- High cardinality filter columns
- Medium to large tables (10-100GB, 10-100M rows)
- Evolving query patterns
- Works well standalone, or combined with partitioning for larger tables



File maintenance

- Use Spark commands to maintain your delta files
- OPTIMIZE
 - Merges smaller files into larger files
 - Improves scan speed and reduces read overhead
 - Schedule after frequent inserts/updates, or weekly
- VACUUM
 - Deletes obsolete files not in the Delta log
 - Default retention = 7 days
 - VACUUM frees up storage
 - Run after OPTIMIZE or heavy deletes
- Target 128-1024 MB per file



Live benchmarks





Demo time

Wrapping up



Conclusion

- Performance in Fabric delta lake is about file layout more than just compute power
- Partitioning can help very large tables, if query patterns are predictable
- Clustering improves file skipping in a more flexible way and for smaller tables
- Partitioning and clustering can be used together as well when needed



Recommendations

- Use partitioning when:
 - Filters are a fixed, low cardinality column (region, date, ...)
 - Table is very large (100M+ rows)
 - Query patterns are predictable or fixed
- Use liquid clustering when:
 - Filters and query patterns evolve over time
 - Tables are larger (10M-100M rows order of magnitude)
- Below 10M rows, don't bother 😊
- Use EXPLAIN and inputFiles() to verify skipping behaviour
- Use regular OPTIMIZE commands (weekly maintenance job) to control files



Session Feedback



Event Feedback

DATA:Scotland